TransferIa · Documentos de cambios

GPU o CPU: los providers de CUDA

Entrega 2 · Ajustes de CUDA en los tres modelos

Commits: `3e3c268` … `5351e26` (8)

Fechas: 11 al 13 de octubre de 2026

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. El concepto: qué es un provider 2
2.1 Probado en tu PC 3
3. Las tres formas que se probaron 3
3.1 Solo CPU (11 oct, primeros commits) 3
3.2 CUDA con opciones (11 oct) 3
3.3 CUDA simple (13 oct, estado final) 4
4. Cómo se comprobó 4
5. Cómo lo pruebas tú 4
6. Lo que aprendimos 4
Glosario 5

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

Entre el 11 y el 13 de octubre se hicieron **8 commits** que cambian una sola cosa: en qué procesador se ejecutan los tres modelos (detector, cambio de cara y oclusión). Todos tocan la misma línea: la lista de **providers**.

Commit	Fecha	Archivo	Qué hizo
`3e3c268`	11 oct	`core/xseg.py`	Oclusión: solo CPU
`d2ee51d`	11 oct	`core/Face_Analyzer.py`	Detector: solo CPU
`a07e9b6`	11 oct	`core/Face_Analyzer.py`	Detector: CUDA con opciones + CPU
`6e76553`	11 oct	`core/Face_Swaper.py`	Cambio de cara: CUDA con opciones + CPU
`bc2b2c8`	11 oct	`core/xseg.py`	Oclusión: CUDA con opciones + CPU
`8a3e40f`	13 oct	`core/Face_Analyzer.py`	Detector: CUDA simple + CPU
`ff50952`	13 oct	`core/xseg.py`	Oclusión: CUDA simple + CPU
`5351e26`	13 oct	`core/Face_Swaper.py`	Cambio de cara: CUDA simple + CPU

> **Importante: Resultado neto:** al terminar, los tres archivos quedaron con la **misma** lista que el original (`["CUDAExecutionProvider", "CPUExecutionProvider"]`), solo escrita en varias líneas y con un comentario. El comportamiento del programa es idéntico al del 21 de agosto. Se comprobó con `git diff -w 85ea6f0 5351e26`.

> **Nota:** Estos commits los hiciste tú, varios desde la web de GitHub. El *por qué* no quedó escrito; la sección 3 reconstruye lo más probable a partir del código.

2. El concepto: qué es un provider

Analogía: un modelo ONNX es una receta y onnxruntime es la cocina. Un **provider** es el cocinero: `CPUExecutionProvider` cocina con el procesador (siempre está), `CUDAExecutionProvider` cocina con

una tarjeta NVIDIA (mucho más rápido, pero solo si están instalados el driver, CUDA, cuDNN y el paquete `onnxruntime-gpu`).

La lista se lee **en orden de preferencia**: onnxruntime intenta el primero y, si no puede, usa el siguiente.

```
ort.InferenceSession("models/face_occluder.onnx",
                     providers=["CUDAExecutionProvider", "CPUExecutionProvider"])
#                               1º si hay GPU NVIDIA       2º respaldo, siempre funciona
```

2.1 Probado en tu PC

Se pidió CUDA primero y se preguntó a onnxruntime qué usó de verdad:

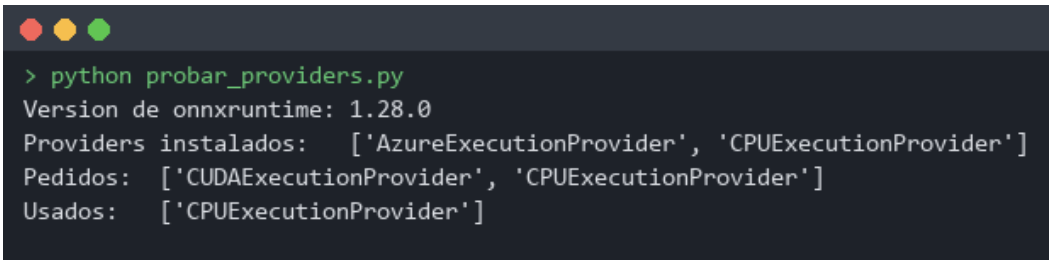


Figura 1. *En este PC no hay provider de CUDA instalado: se pide CUDA y se usa la CPU*

Qué hay que mirar: **Providers instalados** no incluye CUDA (el paquete instalado es `onnxruntime` 1.28, el de CPU). Por eso, aunque el código pida CUDA, **Usados** es solo la CPU. onnxruntime además muestra un aviso: *"Specified provider 'CUDAExecutionProvider' is not in available provider names"*. No es un error: es el respaldo funcionando.

3. Las tres formas que se probaron

3.1 Solo CPU (11 oct, primeros commits)

Antes	Después
providers=['CUDAExecutionProvider', 'CPUExecutionProvider']	providers=['CPUExecutionProvider']

Para qué sirve: forzar la CPU. Es útil si la GPU da errores (por ejemplo, falta de memoria de video o una versión de CUDA incompatible), porque elimina la GPU de la ecuación. **Costo:** en un PC con GPU, todo va varias veces más lento.

3.2 CUDA con opciones (11 oct)

```
providers = [
    ("CUDAExecutionProvider", {
        "device_id": 0,                       # qué tarjeta usar si hay varias
        "cudnn_conv_algo_search": "DEFAULT",  # no probar todos los algoritmos al arrancar
        "cudnn_conv_use_max_workspace": "0"   # no reservar memoria extra de la GPU
    }),
    "CPUExecutionProvider"
]
```

Un provider puede ir como **tupla (nombre, opciones)**. Estas tres opciones apuntan a lo mismo: **arrancar más rápido y gastar menos memoria de video**. Por defecto, cuDNN prueba varios algoritmos la primera vez (`EXHAUSTIVE`) y reserva memoria de trabajo grande; con `DEFAULT` y `0` se evita. Encaja con un entorno de GPU con poca memoria, como Colab gratis.

3.3 CUDA simple (13 oct, estado final)

```
providers = ["CUDAExecutionProvider",
             "CPUExecutionProvider"]
```

Se quitaron las opciones y se volvió a la forma por defecto, igual a la del original.

Forma	Ventaja	Desventaja	Cuándo usarla
Solo CPU	Siempre funciona igual	Lenta con GPU disponible	Para descartar problemas de GPU
CUDA + opciones	Menos memoria y arranque rápido	Puede ser algo más lenta en el proceso	GPU con poca memoria
CUDA simple	La más rápida en GPU, sin configuración	Usa más memoria de video	Uso normal (estado final)

4. Cómo se comprobó

1. Se compararon las versiones `85ea6f0` (antes) y `5351e26` (después) ignorando espacios: solo cambian el formato y un comentario.
2. Se ejecutó la versión `5351e26` con las caras de IA: funciona igual que la original (ver entrega 3, figuras de la versión anterior).
3. Se comprobó qué provider se usa realmente en este PC (figura 1): la CPU.

> **Importante:** No se pudo probar con GPU: este PC no tiene `onnxruntime-gpu` ni CUDA. Las opciones de la sección 3.2 se explican según la documentación de onnxruntime, no por una medición propia.

5. Cómo lo pruebas tú

1. Abre una consola de Python en la carpeta de TransferIa.
2. Escribe `import onnxruntime as ort; print(ort.get_available_providers())`.
3. Si ves `CUDAExecutionProvider`, la GPU está lista; si no, el programa usará la CPU aunque el código pida CUDA.
4. Para usar la GPU en Windows: `pip uninstall onnxruntime` y luego `pip install onnxruntime-gpu`, además del driver NVIDIA y CUDA/cuDNN compatibles con esa versión.

> **Consejo:** Tener instalados a la vez `onnxruntime` y `onnxruntime-gpu` es una causa común de que la GPU "no se vea": Python carga uno de los dos y puede ser el de CPU.

6. Lo que aprendimos

- La lista de providers es una lista de preferencia con respaldo: pedir CUDA nunca rompe el programa en un PC sin GPU.
- Para saber qué se usa de verdad, pregunta a la sesión: `session.get_providers()`.
- Muchos commits pueden sumar cero: comparar el inicio y el final (`git diff A B`) muestra el efecto real.
- La misma lista está repetida en tres archivos (y en `core/Mejorador.py`): una sola constante compartida permitiría cambiarla en un solo lugar. Hoy sigue repetida.

Glosario

Término	Significado
onnxruntime	El motor que ejecuta los modelos .onnx.
Provider	El "motor de cálculo" que usa onnxruntime: CPU, CUDA (NVIDIA), DirectML, etc.
CUDA / cuDNN	Las librerías de NVIDIA para calcular en la GPU; cuDNN es la parte específica de redes neuronales.
Memoria de video (VRAM)	La memoria propia de la tarjeta gráfica; si se llena, el modelo no puede correr en GPU.